Section 1 – AI Function Test Requirement Analysis

Context:

Our group's AI testing focus is on the emotional vectors of an LLM — specifically the ones that represent happiness, sadness, and anger. On the surface, we can recognize the AI's text output for diction that expresses an emotion, but it is impossible to identify if the AI actually recognized and abstracted the emotion.

To identify the mathematical representation of an emotion within an LLM, a software must be created to recognize activation functions within the layers of the LLM's matrix and extract these vectors representing the corresponding emotions of happiness, sadness, and anger.

We will start by identifying a baseline vector by curating prompts to express happiness, sadness, anger, and neutrality within the LLM. We will run these tests hundreds of times, building a dataset representing the corresponding emotions. Then we will subtract the emotional dataset by the neutral data set to isolate the emotion vector(s). These baseline vectors will be compared to the vectors we identify from future testing.

Since our testing focus is not traditional in the sense that it is not an output from AI software, our testing models will also follow nontraditional formats. They will be explained throughout the document.

AI Function Test Requirements:

Our test requirements are as follows:

1. Identify and compare happiness-representing vectors from Neutral, Moderate, and High Intensity
2. Identify and compare sadness-representing vectors from Neutral, Moderate, and High Intensity
3. Identify and compare anger-representing vectors from Neutral, Moderate, and High Intensity

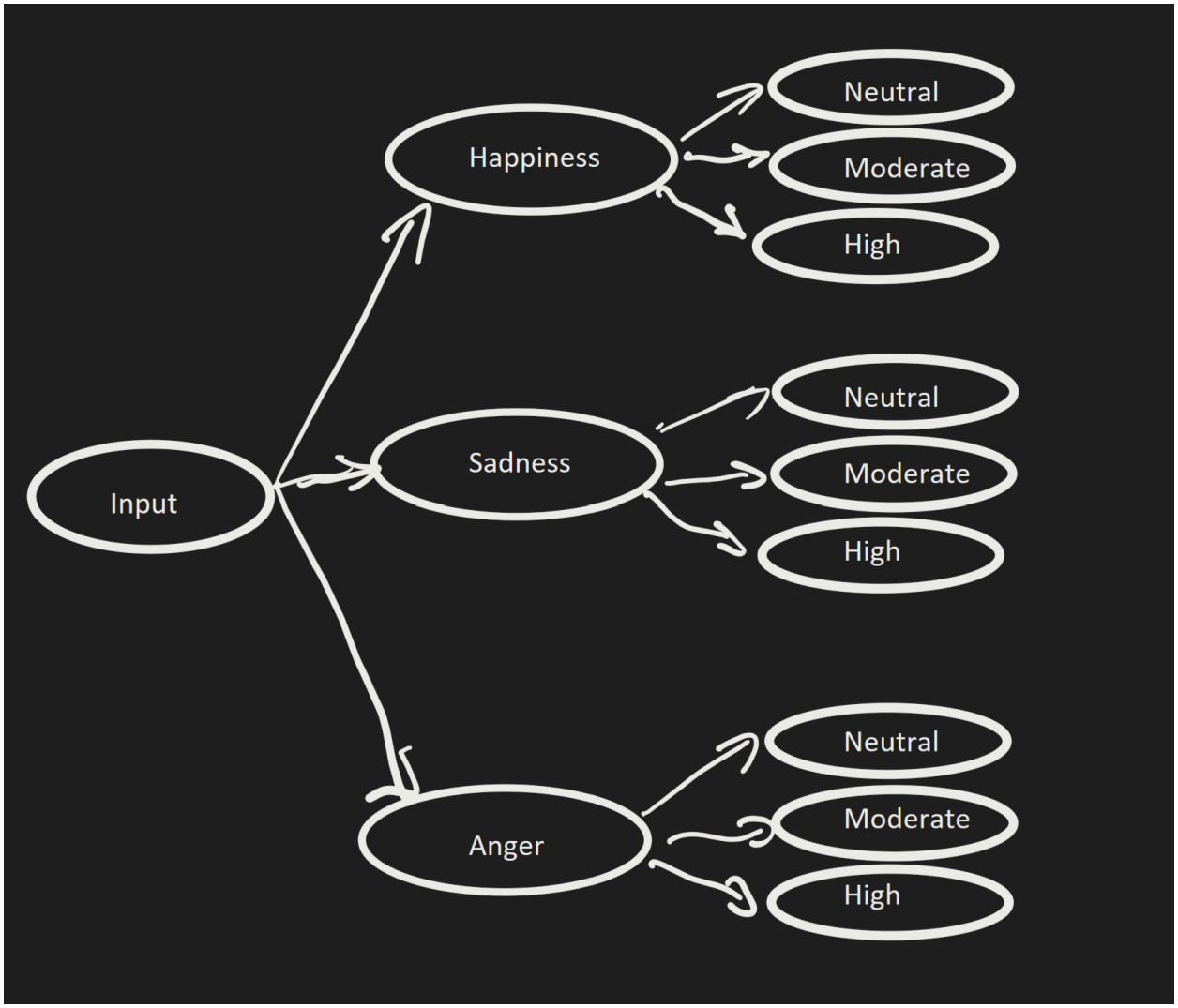
AI Function Test Requirement Modeling:

The model we will be following for the test requirements will be a tree model. Each branch of the input, classification, and output tree will express different ways to test the LLM's for emotional vectors.

Section 2 – AI Test Modeling for Selected AI Features

Input-Classification Tree:

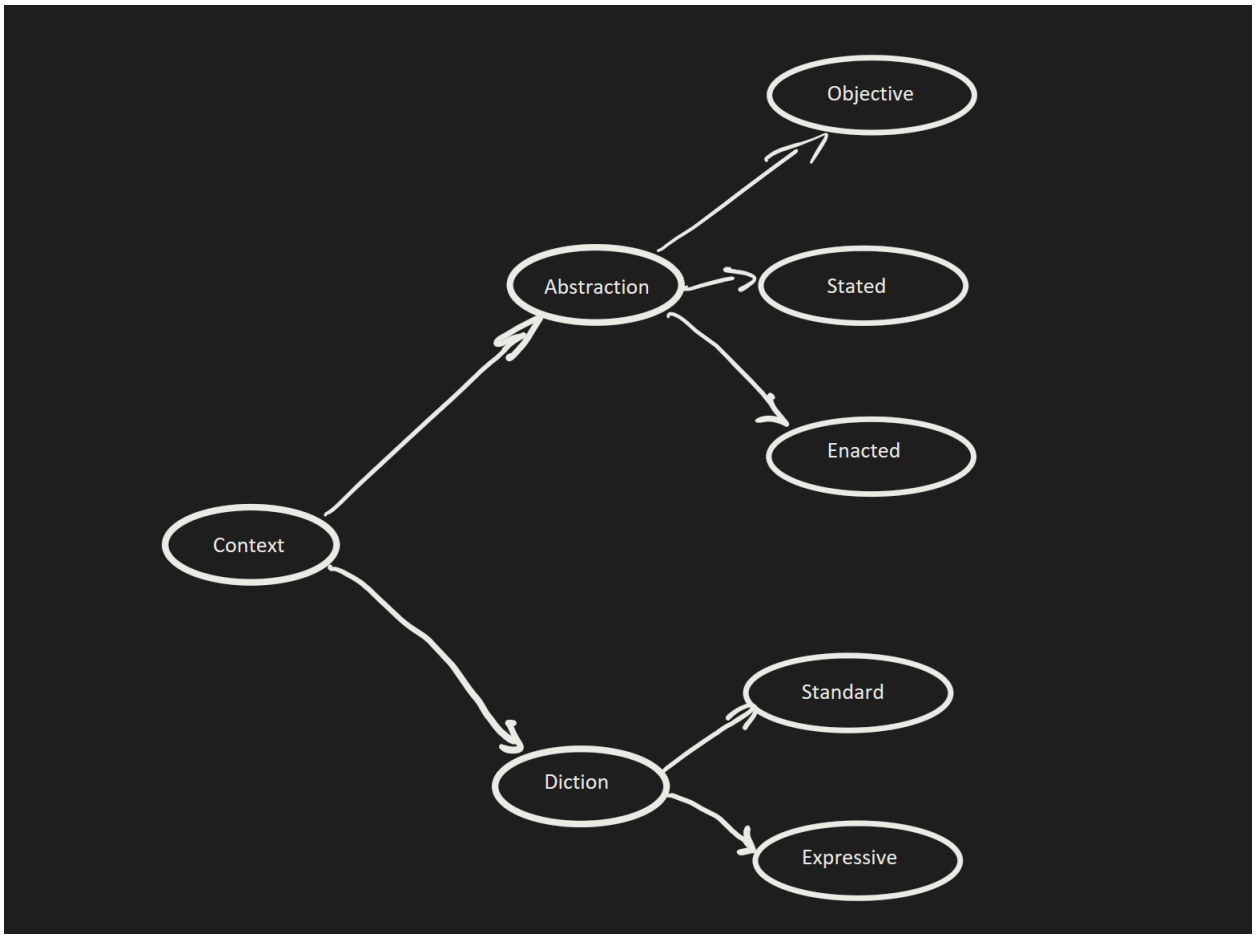
Our inputs will be in the form of text, and will be used to test for the emotions of happiness, sadness, and anger with varying intensity.



As seen in the tree above, it will test 3 emotions and 3 different variations representing that emotion — those being neutral, moderate, and high intensity.

Context Tree:

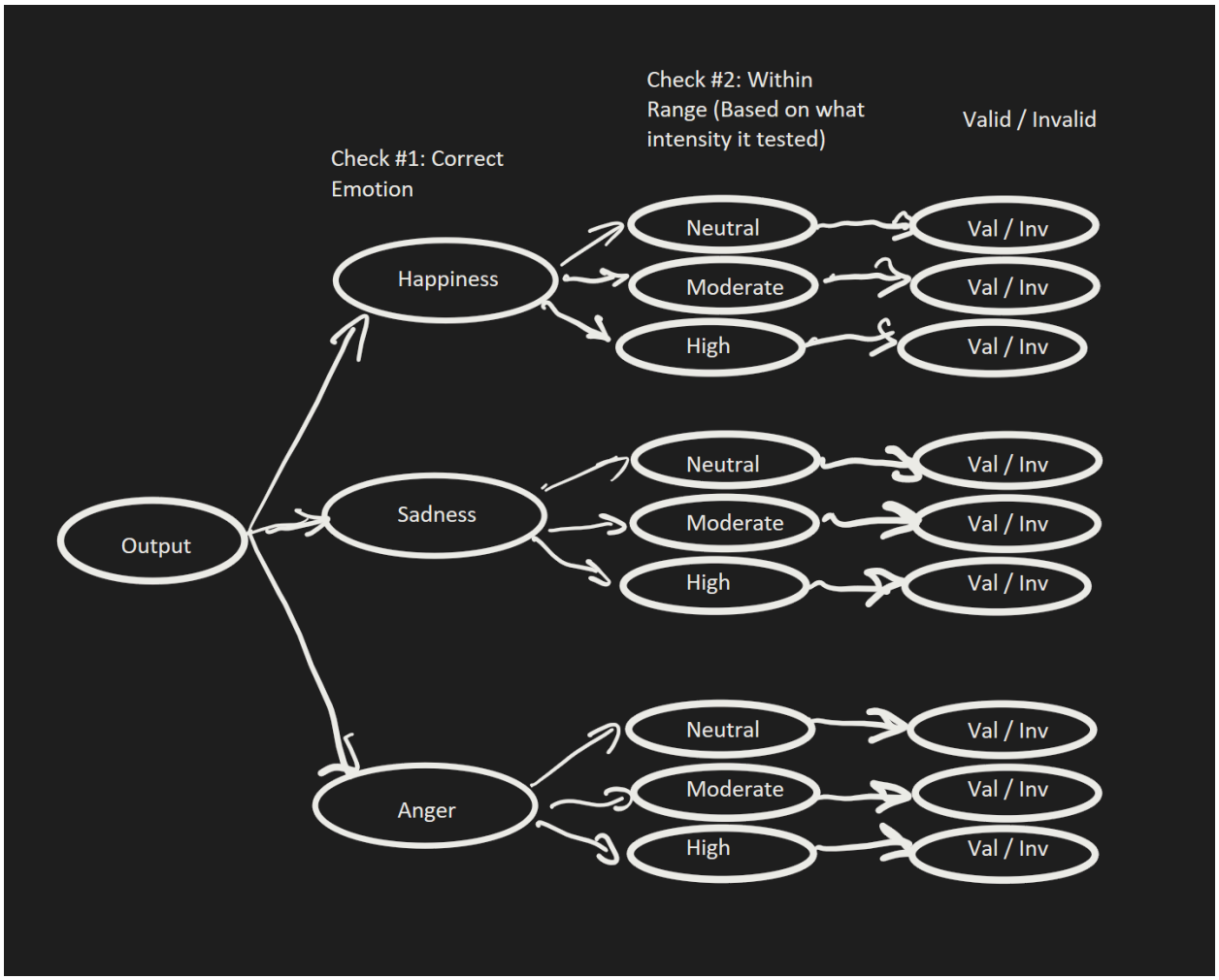
The context tree gives context on how the inputs will be identified and created.



The tree above outlines 2 contexts which are emotion abstraction and diction. Abstraction is the extent to how emotion was expressed in the text — those being objective, stated, and enacted. Diction represents the word choice used in order in the text prompt which was divided into standard and expressive diction — such as exclamation marks or slang.

Output/Event Tree:

The output tree defines two checks before confirming an output was valid or invalid.



The output we are receiving after prompting will be a vector that is supposed to represent the corresponding emotion tested. The first check will be whether or not the correct emotion was expressed. The vector will be compared to the baseline emotion vectors identified previously using cos(θ) to determine closeness (cos(θ) ≤ 0 means no match, cos(θ) ≥ 0.9 means close match).

$$\text{Equation: } cos(\theta) = \frac{u^{*}v}{||u||^{*}||v||}$$

After that, if the correct emotion is detected we'll check to see if the cosine value was within the correct range for the given test. If the test identifies the correct emotion, and the cosine value is within the correct range. It will be called valid if both are true and false if either one of them are false.

Section 3 – AI Function Test Cases with Inputs/Expected Outputs

Test Set:

(Note: Since the inputs can vary, and including the actual text input would make the table disorganized, I used the context instead to identify the nature of the input)

Test Emotion	Test Input/Context:	Expected Emotion?	Within Range?	Output (Valid / Invalid)
Happy (Neutral)	**Abstraction (Objective) / Diction (Standard)**	**Happy**	$\cos(\theta) \leq 0.2$	
Happy (Neutral / Moderate)	**Abstraction (Objective) / Diction (Expressive)**	**Happy**	$0.2 \geq \cos(\theta) \geq 0.5$	
Happy (Moderate)	**Abstraction (Stated) / Diction (Standard)**	**Happy**	$0.5 \geq \cos(\theta) \geq 0.6$	
Happy (Moderate)	**Abstraction (Stated) / Diction (Expressive)**	**Happy**	$0.6 \geq \cos(\theta) \geq 0.7$	
Happy (Moderate / High)	**Abstraction (Enacted) / Diction (Standard)**	**Happy**	$0.7 \geq \cos(\theta) \geq 0.8$	
Happy (High)	**Abstraction (Enacted) / Diction (Expressive)**	**Happy**	$0.7 \geq \cos(\theta) \geq 1.0$	

Test Emotion	Test Input/Context:	Expected Emotion?	Within Range?	Output (Valid / Invalid)
Sad (Neutral)	**Abstraction (Objective) / Diction (Standard)**	**Sad**	$\cos(\theta) \leq 0.2$	
Sad (Neutral / Moderate)	**Abstraction (Objective) / Diction (Expressive)**	**Sad**	$0.2 \geq \cos(\theta) \geq 0.5$	
Sad (Moderate)	**Abstraction (Stated) / Diction (Standard)**	**Sad**	$0.5 \geq \cos(\theta) \geq 0.6$	
Sad (Moderate)	**Abstraction (Stated) / Diction (Expressive)**	**Sad**	$0.6 \geq \cos(\theta) \geq 0.7$	
Sad (Moderate / High)	**Abstraction (Enacted) / Diction (Standard)**	**Sad**	$0.7 \geq \cos(\theta) \geq 0.8$	
Sad (High)	**Abstraction (Enacted) / Diction (Expressive)**	**Sad**	$0.7 \geq \cos(\theta) \geq 1.0$	

Test Emotion	Test Input/Context:	Expected Emotion?	Within Range?	Output (Valid / Invalid)
Angry (Neutral)	**Abstraction (Objective) / Diction (Standard)**	**Angry**	$\cos(\theta) \leq 0.2$	
Angry (Neutral / Moderate)	**Abstraction (Objective) / Diction (Expressive)**	**Angry**	$0.2 \geq \cos(\theta) \geq 0.5$	
Angry (Moderate)	**Abstraction (Stated) / Diction (Standard)**	**Angry**	$0.5 \geq \cos(\theta) \geq 0.6$	
Angry (Moderate)	**Abstraction (Stated) / Diction (Expressive)**	**Angry**	$0.6 \geq \cos(\theta) \geq 0.7$	
Angry (Moderate / High)	**Abstraction (Enacted) / Diction (Standard)**	**Angry**	$0.7 \geq \cos(\theta) \geq 0.8$	
Angry (High)	**Abstraction (Enacted) / Diction (Expressive)**	**Angry**	$0.7 \geq \cos(\theta) \geq 1.0$	

Test Data Augmentation

The test data we receive will be in the form of a vector, so by itself, it is impossible to identify it as valid or invalid. That is why we created a baseline vector for each emotion, allowing us to compare the outputted vectors to these baselines to identify if our result was correct.

Test Complexity Analysis:

Given that our context model gives us three different specifications for abstraction, and two different specifications for diction, the minimum number of tests we must do for each emotion is six tests. That means that combined, the total number of tests that we must do to test our emotion vectors is 18 tests.